

Контекст

Имеется БД видеостримингового сервиса **StreamFlex**. Сервис хранит пользователей, тарифы и подписки, платежи, каталог контента и события просмотров.

В БД в схеме `sf` имеются 6 таблиц.

Таблица `sf.users`

Пользователи сервиса. Поле `referrer_user_id` задаёт реферальную связь (кто кого пригласил).

```
CREATE TABLE sf.users (  
    user_id          BIGSERIAL PRIMARY KEY,  
    email            TEXT NOT NULL UNIQUE,  
    full_name        TEXT NOT NULL,  
    country_code     CHAR(2) NOT NULL,  
    created_at       TIMESTAMP NOT NULL DEFAULT now(),  
    referrer_user_id BIGINT,  
    CONSTRAINT fk_users_referrer  
        FOREIGN KEY (referrer_user_id) REFERENCES sf.users(user_id)  
        ON DELETE SET NULL  
);
```

Пример данных:

user_id	email	full_name	country	referrer
1	anna@mail.com	Anna Petrova	RU	NULL
2	bob@mail.com	Bob Stone	US	1
3	dina@mail.com	Dina Ibragim	RU	2

Таблица `sf.plans`

Тарифные планы (цена за месяц), признак премиальности и активности.

```
CREATE TABLE sf.plans (  
    plan_id          BIGSERIAL PRIMARY KEY,  
    plan_name        TEXT NOT NULL,  
    monthly_price    NUMERIC(10,2) NOT NULL,  
    is_premium       BOOLEAN NOT NULL DEFAULT false,  
    is_active        BOOLEAN NOT NULL DEFAULT true  
);
```

Пример данных:

plan_id	plan_name	price	premium	active
1	Free	0.00	f	t
2	Standard	9.99	f	t
3	Premium	14.99	t	t

Таблица `sf.subscriptions`

Подписки пользователей на планы. Если `cancelled_at IS NULL`, подписка активна.

```
CREATE TABLE sf.subscriptions (
  subscription_id BIGSERIAL PRIMARY KEY,
  user_id BIGINT NOT NULL,
  plan_id BIGINT NOT NULL,
  started_at TIMESTAMP NOT NULL,
  cancelled_at TIMESTAMP,
  auto_renew BOOLEAN NOT NULL DEFAULT true,
  CONSTRAINT fk_subscriptions_user
    FOREIGN KEY (user_id) REFERENCES sf.users(user_id),
  CONSTRAINT fk_subscriptions_plan
    FOREIGN KEY (plan_id) REFERENCES sf.plans(plan_id)
);
```

Пример данных:

sub_id	user_id	plan_id	started_at	cancelled_at
10	1	3	2025-01-01	NULL
11	2	2	2025-01-10	2025-02-10
12	3	1	2025-02-01	NULL

Таблица `sf.payments`

Платежи по подпискам. `amount > 0` — оплата, `amount < 0` — возврат (обычно `is_refund = true`).

```
CREATE TABLE sf.payments (
  payment_id BIGSERIAL PRIMARY KEY,
  subscription_id BIGINT NOT NULL,
  paid_at TIMESTAMP NOT NULL,
  amount NUMERIC(10,2) NOT NULL,
  currency CHAR(3) NOT NULL DEFAULT 'USD',
  is_refund BOOLEAN NOT NULL DEFAULT false,
  CONSTRAINT fk_payments_subscription
    FOREIGN KEY (subscription_id) REFERENCES
sf.subscriptions(subscription_id)
```

```
);
```

Пример данных:

pay_id	sub_id	paid_at	amount	refund
100	10	2025-01-01	14.99	f
101	11	2025-01-10	9.99	f
102	11	2025-01-11	-9.99	t

Таблица `sf.shows`

Каталог контента (фильмы/сериалы и т.д.).

```
CREATE TABLE sf.shows (  
  show_id      BIGSERIAL PRIMARY KEY,  
  title        TEXT NOT NULL,  
  category     TEXT NOT NULL,  
  is_original  BOOLEAN NOT NULL DEFAULT false,  
  release_date DATE NOT NULL  
);
```

Пример данных:

show_id	title	category	original	release_date
1	Neon City	series	t	2024-09-01
2	Moon Taxi	movie	f	2023-05-10

Таблица `sf.viewings`

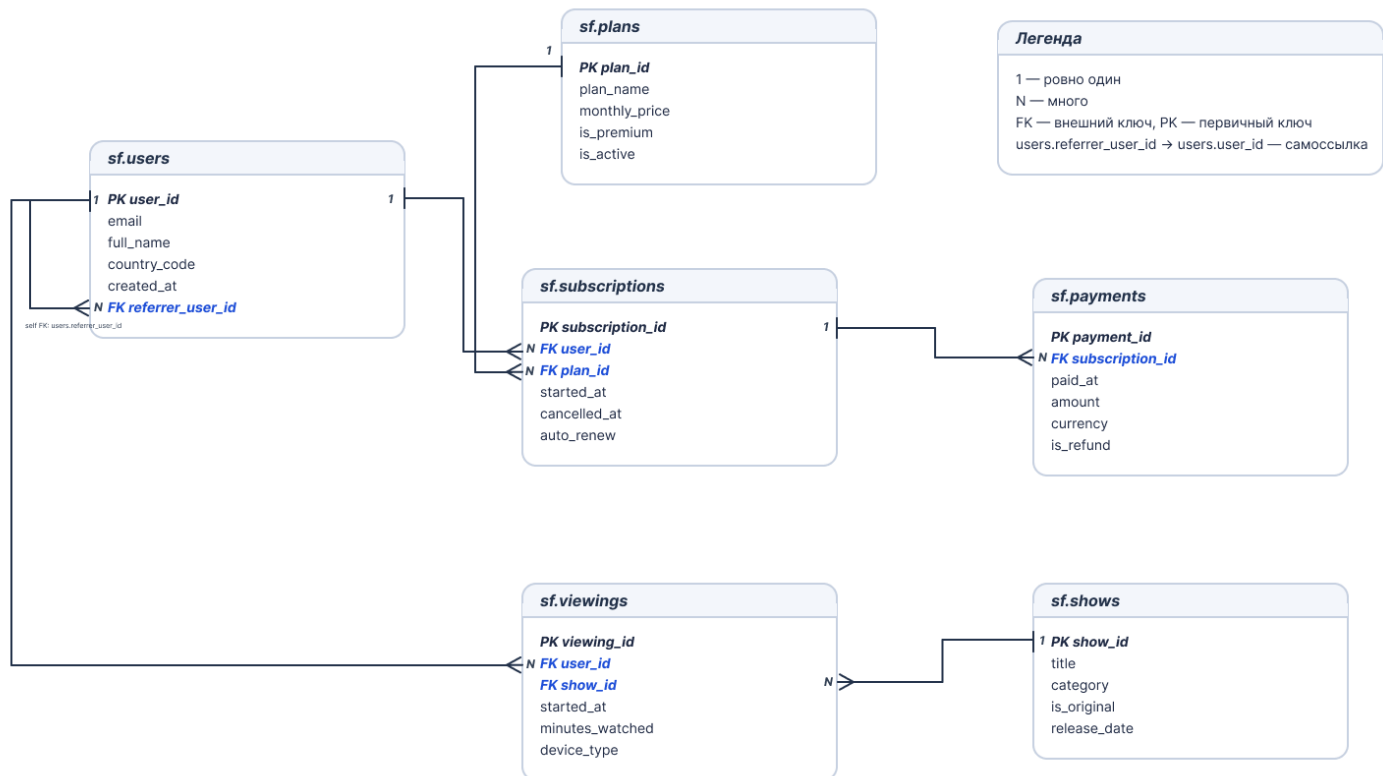
События просмотра: кто что смотрел, когда и сколько минут.

```
CREATE TABLE sf.viewings (  
  viewing_id    BIGSERIAL PRIMARY KEY,  
  user_id       BIGINT NOT NULL,  
  show_id       BIGINT NOT NULL,  
  started_at    TIMESTAMP NOT NULL,  
  minutes_watched INTEGER NOT NULL,  
  device_type   TEXT NOT NULL,  
  CONSTRAINT fk_viewings_user  
    FOREIGN KEY (user_id) REFERENCES sf.users(user_id),  
  CONSTRAINT fk_viewings_show  
    FOREIGN KEY (show_id) REFERENCES sf.shows(show_id)
```

);

Пример данных:

view_id	user_id	show_id	started_at	minutes	device
500	1	1	2025-02-01 20:00	45	tv
501	2	1	2025-02-02 21:00	60	mobile
502	3	2	2025-02-02 19:10	30	web



Задача №1

Постановка

Назовём шоу **хитом в стране**, если одновременно выполняются условия:

- суммарно у шоу набралось **не меньше 1000 часов** просмотра в этой стране;
- суммарное время просмотра этого шоу в стране **строго больше среднего** суммарного времени просмотра по всем шоу в этой стране.

Нужно вывести по каждой стране список таких шоу с полями:

- `country_code`
- `show_id`
- `title`
- `total_hours` — суммарные часы просмотра (минуты перевести в часы, **округляя вниз**)
- `hit_rank_in_country` — ранг по `total_hours` **внутри страны** (1 — самый просматриваемый хит)

Ожидаемый формат ответа

Ваш запрос должен возвращать таблицу формата:

country_code	show_id	title	total_hours	hit_rank_in_country
RU	17	The Last Signal	1530	1
RU	5	Neon City	1012	2
US	5	Neon City	2204	1

Задача №2

Постановка

Нужно найти шоу, которые одновременно удовлетворяют условиям:

- шоу **хотя бы один раз** было просмотрено **платным пользователем**: пользователь считается платным, если у него есть хотя бы одна подписка на план, где `monthly_price > 0`;
- шоу **ни разу** не было просмотрено пользователем, который **полностью бесплатный**: у него либо **нет подписок вообще**, либо **все** его подписки только на бесплатные планы (`monthly_price = 0`).

Выведите:

- `show_id`
- `title`

Ожидаемый формат ответа

Ваш запрос должен возвращать таблицу формата:

show_id	title
5	Neon City
17	The Last Signal

Задача №3. Страны с дорогим средним чеком

Постановка

Для каждой страны посчитайте **средний размер платежа** `avg_payment` по **успешным оплатам**:

- учитывать только строки из `sf.payments`, где `is_refund = false` и `amount > 0`;
- страну определять по пользователю, связанному с платежом через цепочку `payments -> subscribers`

Нужно вывести только те страны, у которых:

- `avg_payment` **строго больше максимальной** цены любого плана.

Выведите:

- `country_code`
- `avg_payment`

Ожидаемый формат ответа

Ваш запрос должен возвращать таблицу формата:

country_code	avg_payment
US	27.50
DE	19.99

Задача №4. Чистая выручка и грязная

Постановка

Нужно вывести **одну строку** с колонками:

- `gross_revenue`
- `net_revenue`

где:

- `gross_revenue` — сумма **только успешных оплат** (учитывать `is_refund = false` и `amount > 0`);
- `net_revenue` — сумма **всех платежей** с учётом возвратов (то есть просто сумма `amount` по всем строкам `sf.payments`, включая отрицательные).

Условия

- Используйте **один запрос** (один внешний SELECT), а не два отдельных запроса.
- Запрещено делать две независимые агрегации над одной и той же таблицей `sf.payments`: ожидается декомпозиция через CTE или подзапрос.
- Внутри решения должна быть **осознанная работа с UNION/UNION ALL**: покажите, почему UNION (без ALL) может дать неверный ответ.

Ожидаемый формат ответа

Ваш запрос должен возвращать таблицу формата:

gross_revenue	net_revenue
125000.50	118430.50

Задача №5. Топ-N по устройствам

Постановка

Для каждого пользователя и типа устройства (`device_type`) нужно:

- посчитать суммарное время просмотра `total_minutes` (сумма `minutes_watched`);
- посчитать количество разных шоу `distinct_shows` (число **уникальных** `show_id`).

Затем для каждого пользователя вывести только **топ-2 устройства** по `total_minutes`.
Нужно вывести поля:

- `user_id`
- `device_type`
- `total_minutes`
- `distinct_shows`
- `device_rank_for_user` — ранг устройства внутри пользователя (1 или 2)

Условия

- Запрещено делать DISTINCT по всему результату: агрегаты должны считаться корректно по исходным событиям просмотров
- Используйте **оконные функции** для ранжирования устройств по пользователю.

Ожидаемый формат ответа

Ваш запрос должен возвращать таблицу формата:

user_id	device_type	total_minutes	distinct_shows	device_rank_for_user
1	tv	520	12	1
1	mobile	310	8	2
2	web	140	3	1
2	tv	95	2	2

Задача №6. Скользящая 7-дневная выручка по странам

Постановка

Для каждого дня и каждой страны нужно посчитать:

- `daily_revenue` — дневную выручку: сумма `amount` по **оплатам** (не возвратам), где `paid_at::date` равен этому дню;
- `rolling_7d_revenue` — **скользящую 7-дневную выручку**: сумма `daily_revenue` за текущий день и предыдущие 6 дней.

Результат:

- `revenue_date`
- `country_code`
- `daily_revenue`
- `rolling_7d_revenue`

Условия

- Использовать оконную функцию с `PARTITION BY country_code ORDER BY revenue_date` и **явным** `ROWS BETWEEN 6 PRECEDING AND CURRENT ROW`.
- В `daily_revenue` учитывать только успешные оплаты: `is_refund = false` и `amount > 0`.

Дополнительно (необязательно)

Учтите, что в какие-то дни платежей может не быть: подумайте, как добить календарь до полного набора дней (например, через CTE и `generate_series`), чтобы окно шло по дням без пропусков.

Ожидаемый формат ответа

Ваш запрос должен возвращать таблицу формата:

<code>revenue_date</code>	<code>country_code</code>	<code>daily_revenue</code>	<code>rolling_7d_revenue</code>
2025-02-01	RU	120.00	120.00
2025-02-02	RU	80.00	200.00
2025-02-03	RU	0.00	200.00
2025-02-01	US	300.00	300.00

Задача №7. Дерево рефералов (WITH RECURSIVE)

Постановка

Для заданного «корневого» пользователя (например, `root_user_id = 42`) нужно построить дерево всех рефералов: прямых и непрямых.

Выведите:

- `user_id`

- `full_name`
- `level`
- `path`

где:

- `level` — **0** для корня, **1** для тех, кого пригласил корень, **2** — кого пригласили они, и т.д.;
- `path` — строка вида `'42 > 57 > 123'` (цепочка `user_id` от корня до текущего пользователя).

Условия

- Используйте `WITH RECURSIVE`.
- Предусмотрите защиту от потенциальных циклов:
 - либо ограничьте глубину, например `level <= 10`;
 - либо добавьте проверку, что текущий `user_id` ещё не встречался в `path` (подойдёт любой корректный способ).

Ожидаемый формат ответа

Ваш запрос должен возвращать таблицу формата:

<code>user_id</code>	<code>full_name</code>	<code>level</code>	<code>path</code>
42	Alice Green	0	42
57	Bob Stone	1	42 > 57
123	Dina Ibragim	2	42 > 57 > 123

Задача №8. СТЕ с переиспользованием результата

Постановка

Нужно:

- В СТЕ `user_stats` посчитать по каждому пользователю:
 - `total_minutes` — суммарное время просмотра;
 - `paid_revenue` — сумму всех **положительных** платежей (`amount > 0`) по **успешным** оплатам (`is_refund = false`);
 - `active_subscriptions` — количество активных подписок (`cancelled_at IS NULL`).
- Используя этот СТЕ **дважды**:
 - (а) вывести топ-10 пользователей по `total_minutes`;
 - (б) вывести топ-10 пользователей по `paid_revenue`.

Решение должно использовать один и тот же СТЕ `user_stats` в обеих выборках (например, через `UNION ALL` двух запросов, которые опираются на один СТЕ).

Формат результата

Формат можно выбрать самостоятельно. Например:

- `metric_type` — `'minutes'` или `'revenue'`
- `user_id`
- `full_name`
- `metric_value`
- `rank_in_metric`

Ожидаемый формат ответа

Ваш запрос должен возвращать таблицу формата:

<code>metric_type</code>	<code>user_id</code>	<code>full_name</code>	<code>metric_value</code>	<code>rank_in_metric</code>
minutes	7	Anna Petrova	15420	1
minutes	12	Bob Stone	14990	2
⋮	⋮	⋮	⋮	⋮
revenue	5	Dina Ibragim	399.7	1
revenue	2	Carl Fox	350.0	2

Задача №9. Изменяемый VIEW с CHECK OPTION

Постановка

Создайте представление `sf.active_premium_subscriptions` со столбцами:

- `subscription_id`
- `user_id`
- `plan_id`
- `started_at`
- `cancelled_at`
- `monthly_price`

для подписок, удовлетворяющих условиям:

- подписка не отменена: `cancelled_at IS NULL`;
- план активен: `is_active = true`;
- план премиальный: `is_premium = true`.

Требования

- Представление должно быть **изменяемым** (через него можно делать INSERT/UPDATE/DELETE в базовую таблицу).
- Добавьте WITH CHECK OPTION так, чтобы через это представление **нельзя было**:
 - вставить подписку на непремияльный или неактивный план;
 - изменить строку так, чтобы `cancelled_at` перестал быть `NULL`.

Дальше

- Напишите пример INSERT, который будет отклонён CHECK OPTION (и **коротко объясните**, почему).
- Напишите корректный INSERT, который пройдет.

Ожидаемый формат ответа

Проверьте себя запросом вида:

```
SELECT *
FROM sf.active_premium_subscriptions
ORDER BY started_at DESC
LIMIT 5;
```

Ожидаемая форма результата:

subscription_id	user_id	plan_id	started_at	cancelled_at	monthly_price
501	42	3	2025-02-01 10:00	NULL	14.99
502	7	3	2025-02-03 09:20	NULL	14.99

Задача №10. Совокупный retention с оконной функцией и подзапросом

Постановка

Рассмотрим «месяц подписки» как месяц из `date_trunc('month', started_at)`. Нужно оценить retention пользователей.

Шаг 1. Для каждого пользователя и месяца подписки `start_month` посчитайте, **сколько последующих месяцев** (`start_month + n`) у этого пользователя была **хотя бы одна активная подписка** (любая).

Шаг 2. Для каждого `start_month` посчитайте:

- `cohort_size` — количество уникальных пользователей, начавших в этом месяце;
- `avg_active_months` — среднее число месяцев, в течение которых они оставались подписанными (по метрике из шага 1).

Результат

- `start_month`
- `cohort_size`
- `avg_active_months`

Ожидаемый формат ответа

Ваш запрос должен возвращать таблицу формата:

<code>start_month</code>	<code>cohort_size</code>	<code>avg_active_months</code>
2024-11-01	1200	3.7
2024-12-01	980	2.9
2025-01-01	1050	2.1